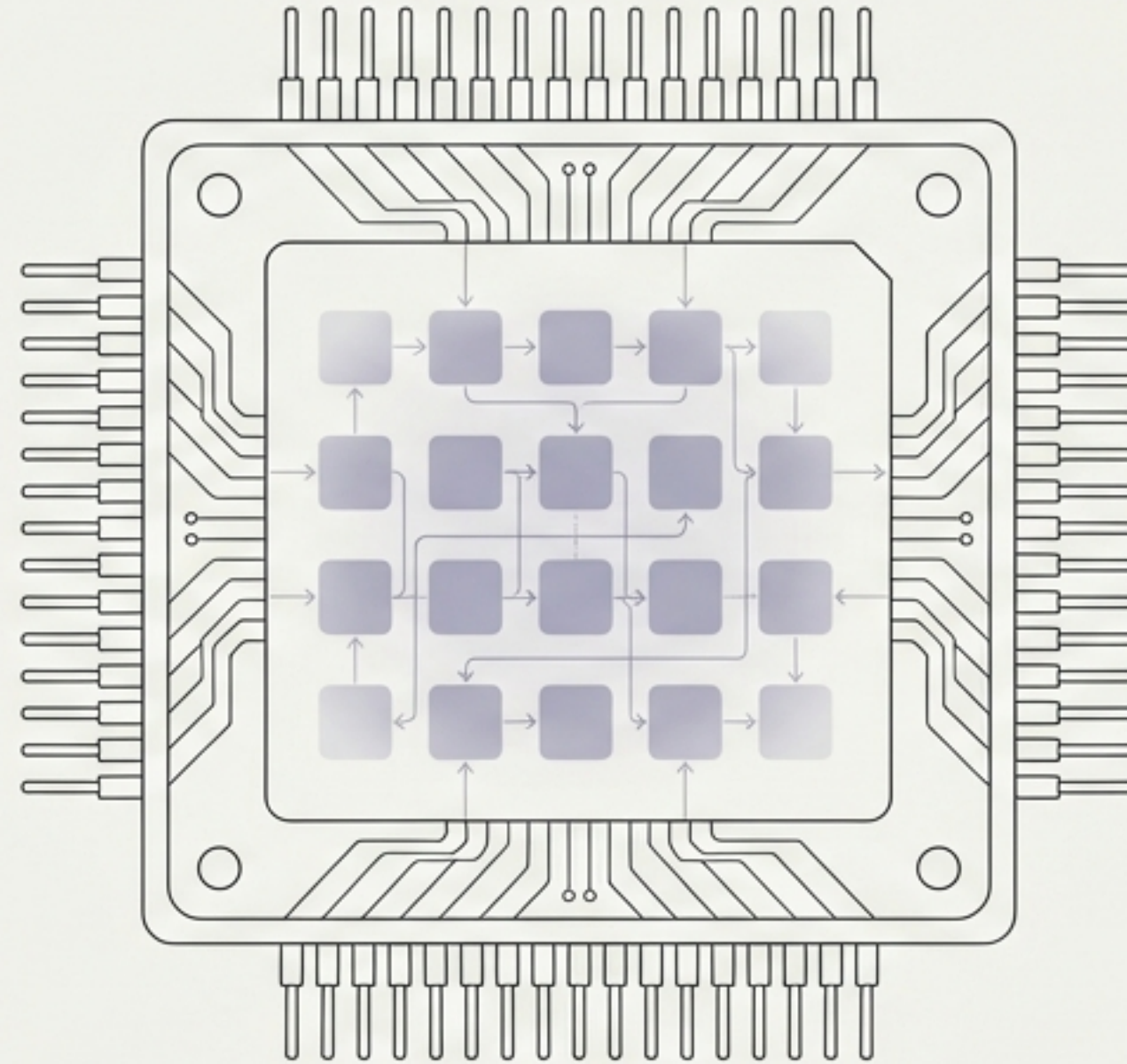


DOC: issues-fs__llm-as-execution-engine
VER: v1.0
STATUS: Practical Guide
DATE: 05 February 2026



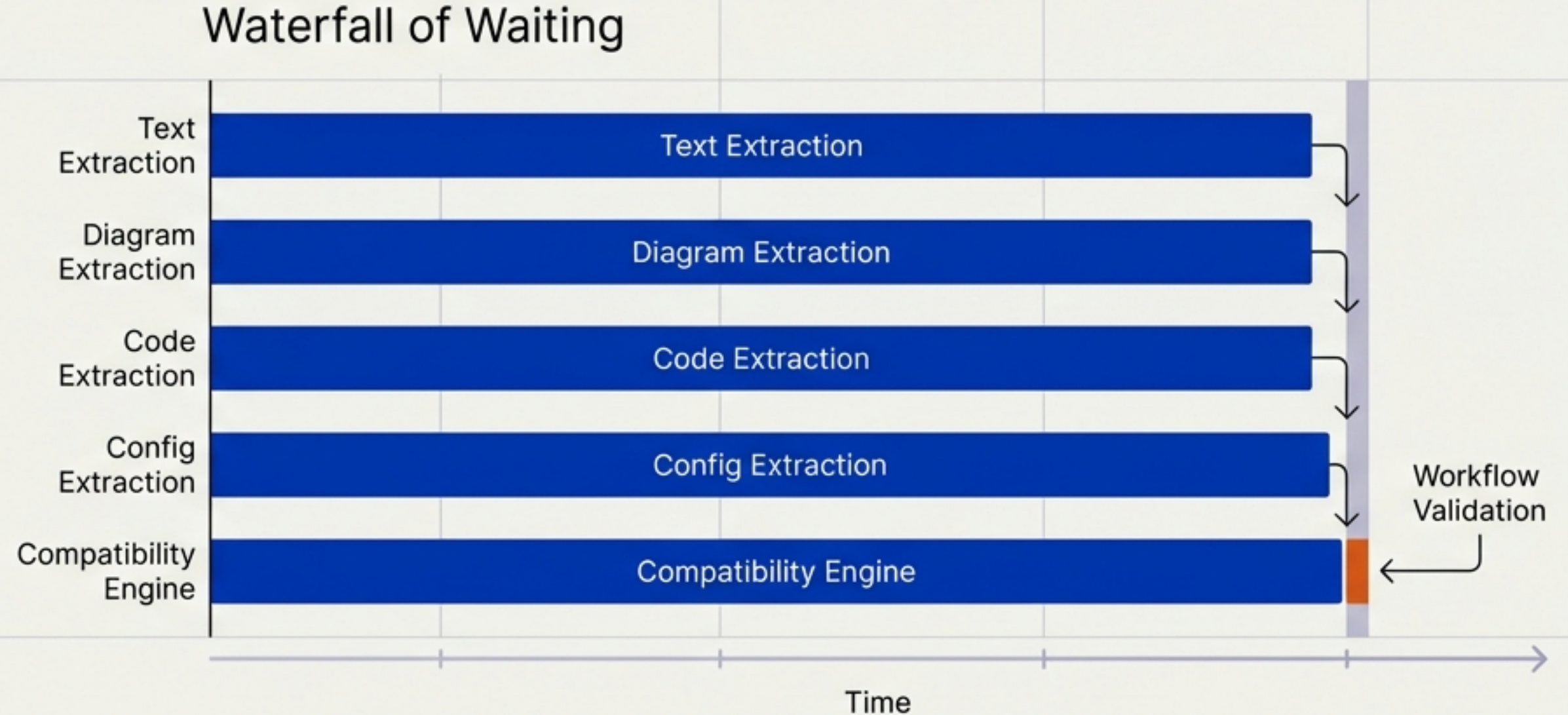
LLM as Execution Engine

Implementing Workflows Before Code Exists

The Bootstrap Problem

To validate the compatibility testing workflow, we theoretically need a massive suite of implemented components.

Waiting for this code creates a blind spot where design errors hide until it is too late.

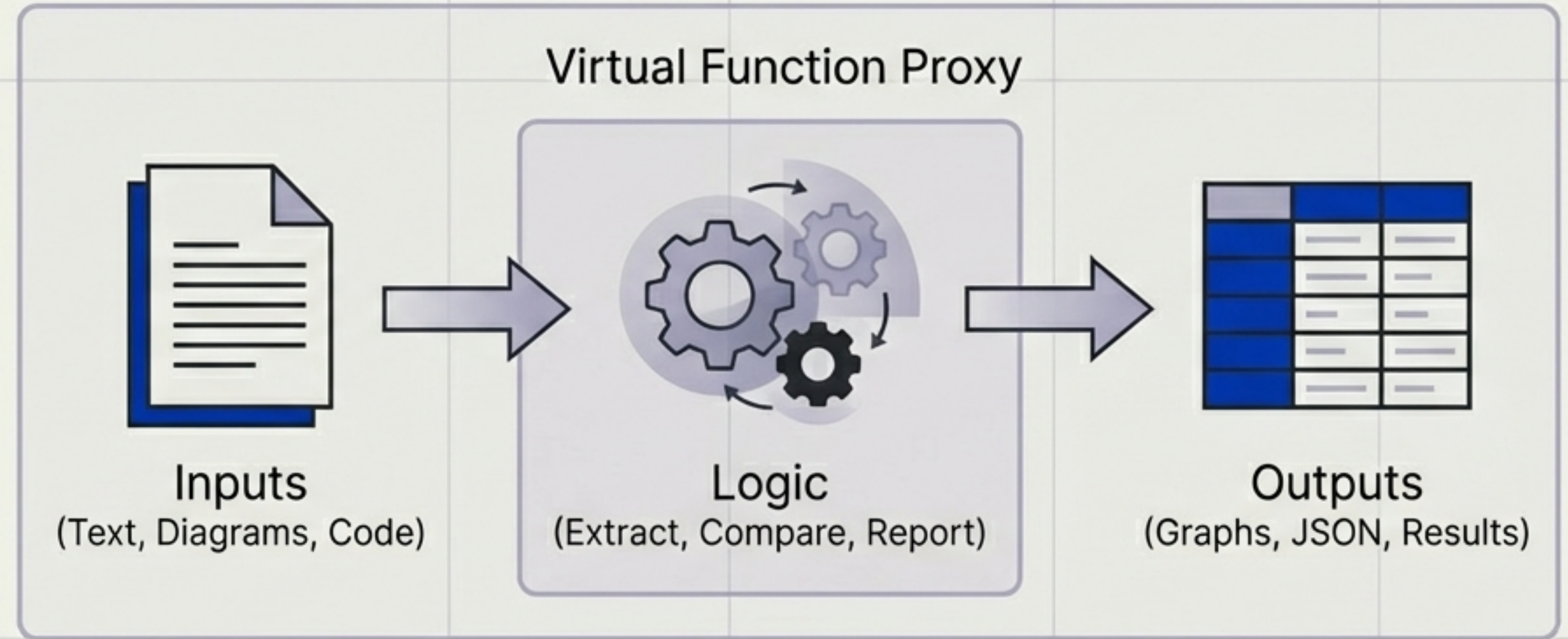


Key Insight: If we wait until all of this is implemented to test the workflow, we validate the design only after the cost of change has peaked.

We Don't Need Code to Execute Logic

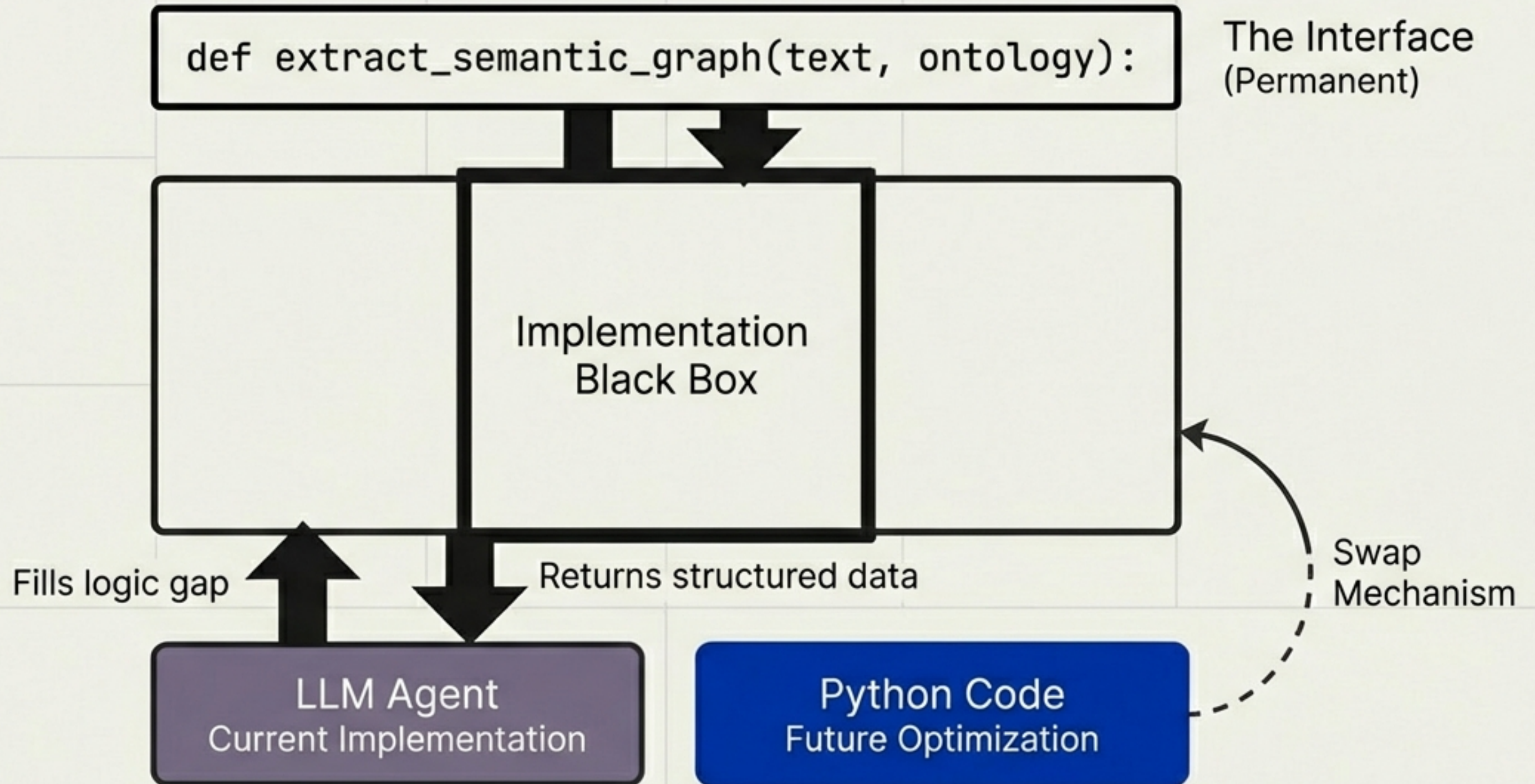
Every function described in our architecture possesses three fundamental attributes.

An LLM acts as a proxy execution engine by satisfying all three immediately.



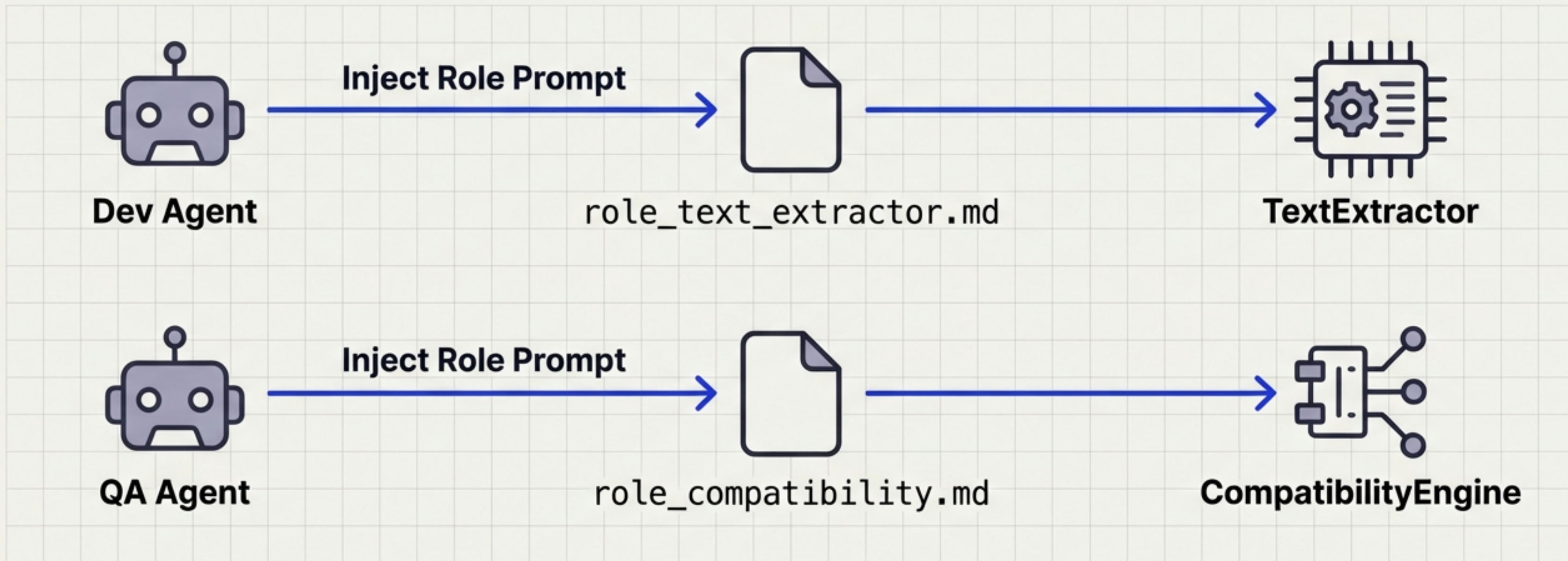
“The layers above do not know—and do not care—whether they are calling compiled Python or an LLM acting as code.”

The Interface Abstraction



Role-Based Agents as Component Proxies

Issues-FS utilizes specific agent personas. By applying 'Role Prompts,' we convert generic agents into specific system components.



The prompt reconfigures the general purpose agent into a specialized architectural building block.

Case Study: The Text Extractor

INPUT

Architecture Doc

```
API Gateway (Kong) –  
The single entry point...  
Handles authentication  
(JWT validation)...  
All external traffic  
must pass through here.
```

PROMPT (LOGIC)

Specification

```
BEHAVIOUR: Parse text to  
identify Components,  
Technologies, Rules.  
ONTOLOGY: Node types  
(Component, Rule); Edge  
types (uses, enforces).  
OUTPUT: JSON following  
SemanticGraph schema.
```

OUTPUT

Structured Data

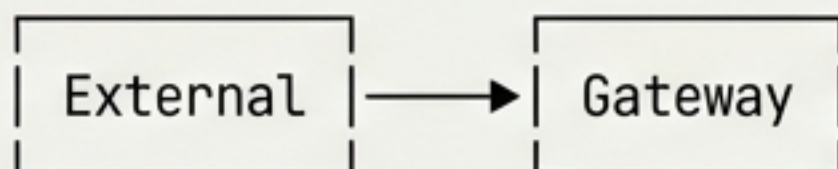
```
{  
  "id": "api_gateway",  
  "type": "Component",  
  "properties": {  
    "auth": "JWT"  
  }  
}
```

The agent received input, followed logic, and produced valid JSON. It acted like code.

Case Study: The Diagram Extractor

INPUT

ASCII Diagram



PROMPT (LOGIC)

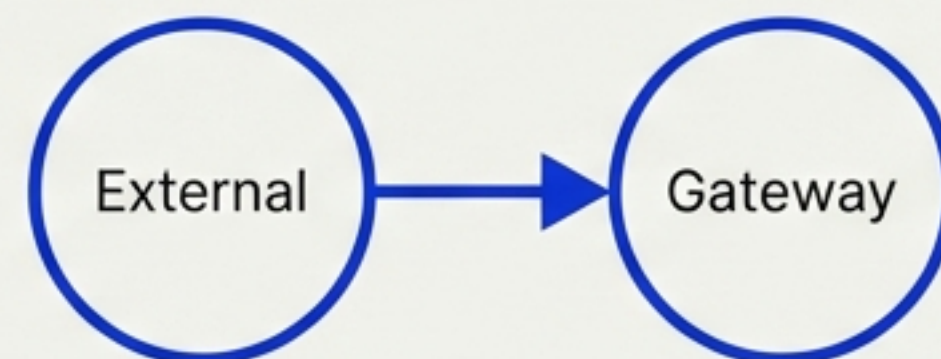
Extraction Rules

Prompt:

1. Identify boxes (Nodes).
2. Identify arrows (Edges).
3. Extract containment & zones.

OUTPUT

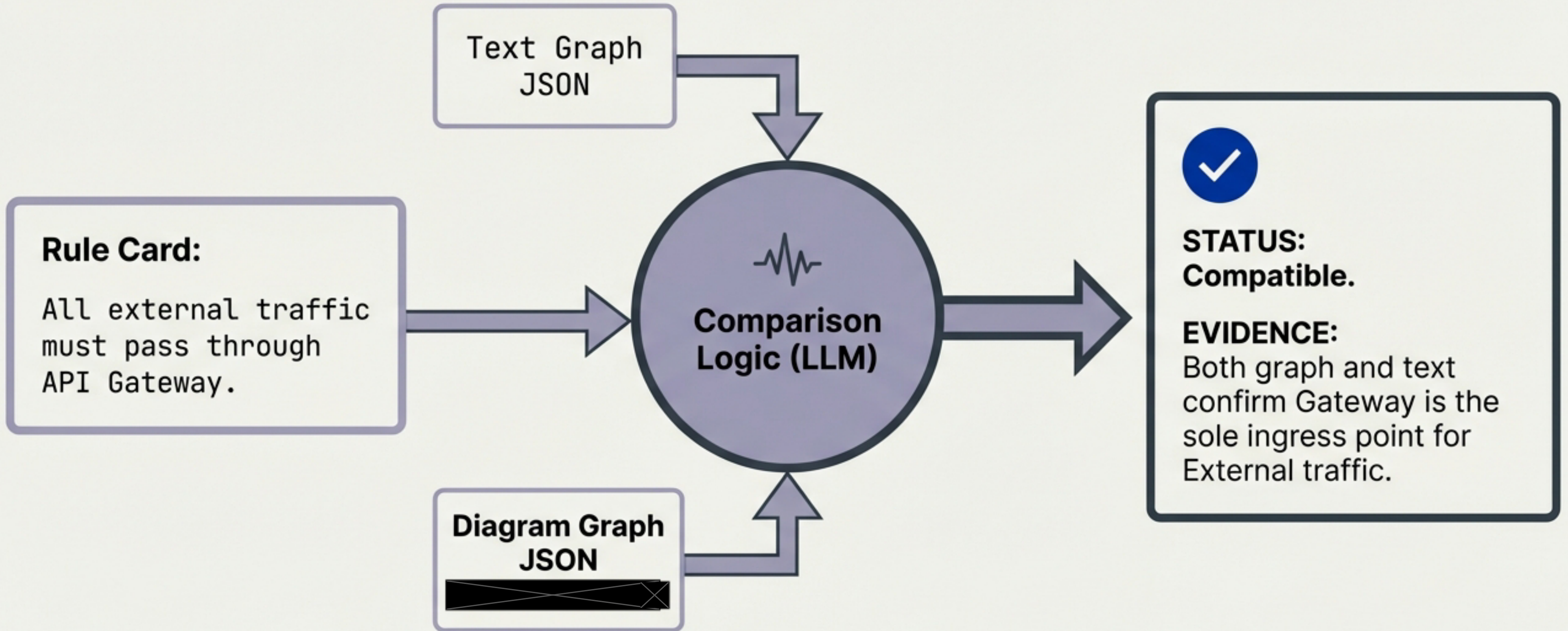
Rendered Graph & JSON



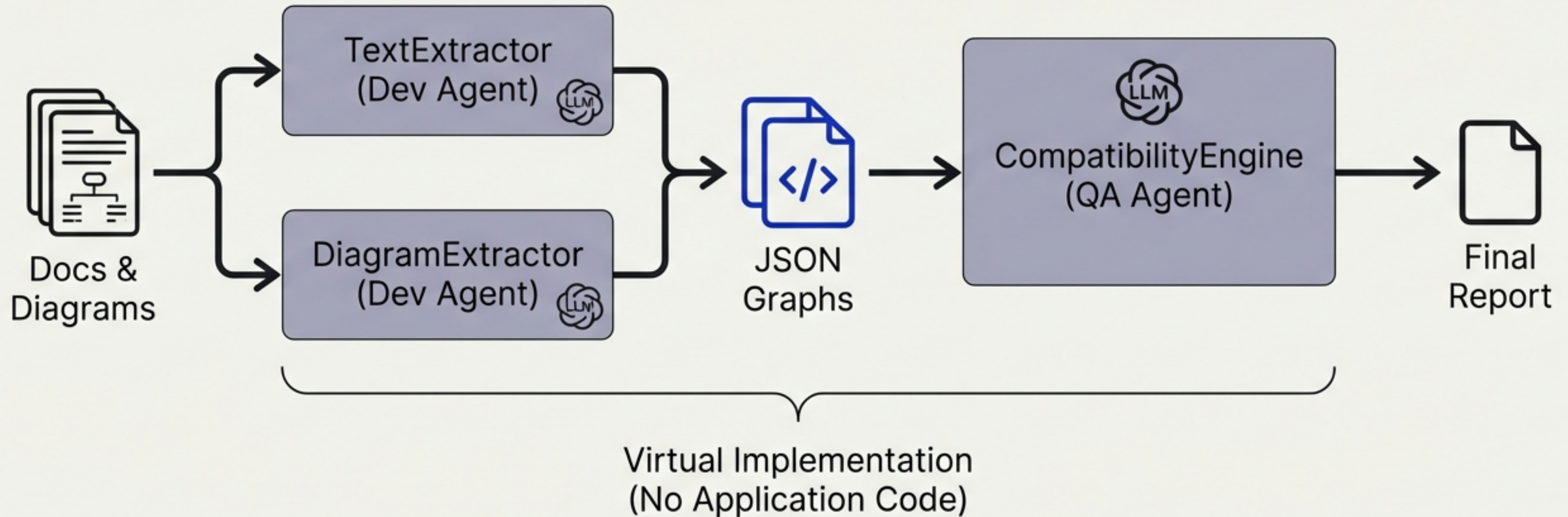
```
{  
  "source": "External",  
  "target": "Gateway"  
}
```

Case Study: The Compatibility Engine

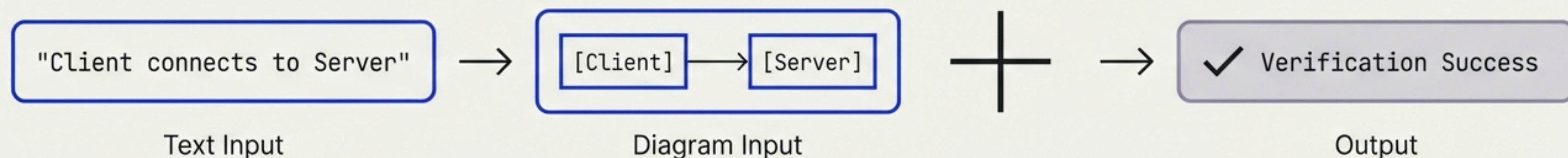
The QA Agent compares the outputs from the Text and Diagram extractors against a defined rule.



Orchestrating the Virtual Workflow

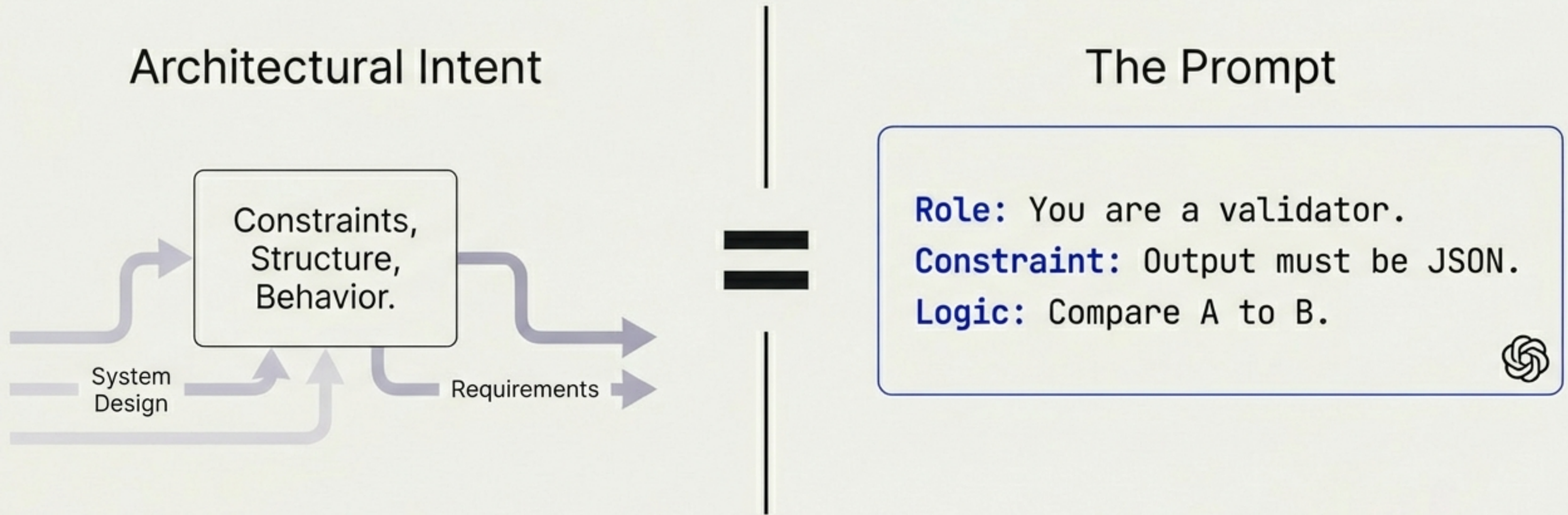


Validating with 'Hello World'



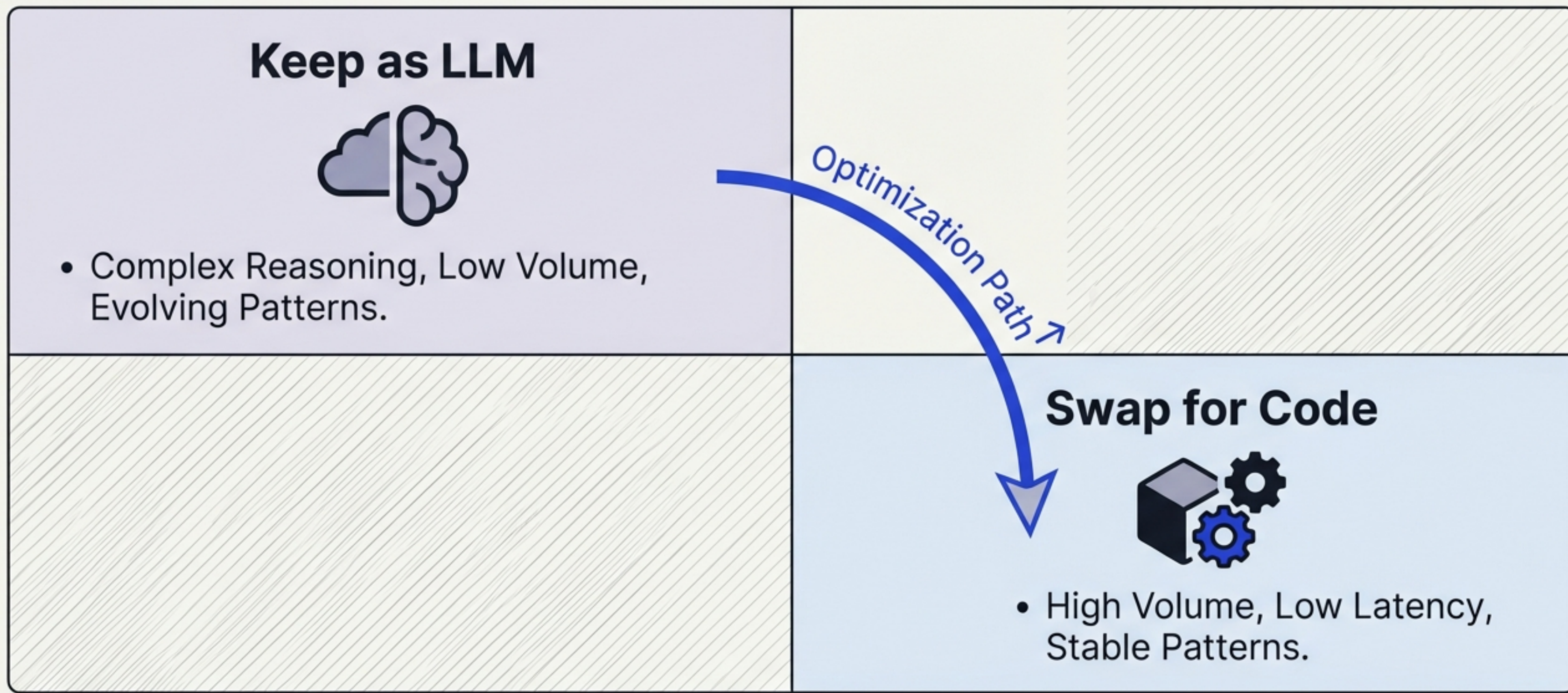
Do not start with the full platform. We validate the virtual workflow with the simplest possible assertion. Once the "Hello World" pipeline holds, we scale to complex architectures.

The Prompt Is the Specification



The prompt-writer is the architect. The LLM is the implementation.
The prompt bridges the gap between intent and execution.

The Gradual Replacement Strategy



Code is merely an optimization for stable patterns.

Benefits of the Virtual Approach



Immediate Validation

Validate architecture docs and diagrams today. Find missing rules and ambiguities now.



Living Documentation

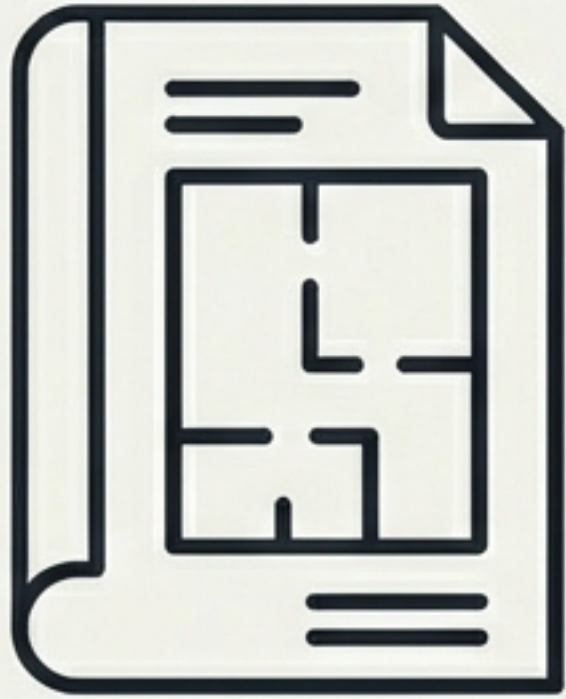
The Role Prompts **are** the documentation. They specify exactly what inputs are expected.



Testable Prompts

Create a feedback loop. If LLM output differs from Code output, the spec is wrong.

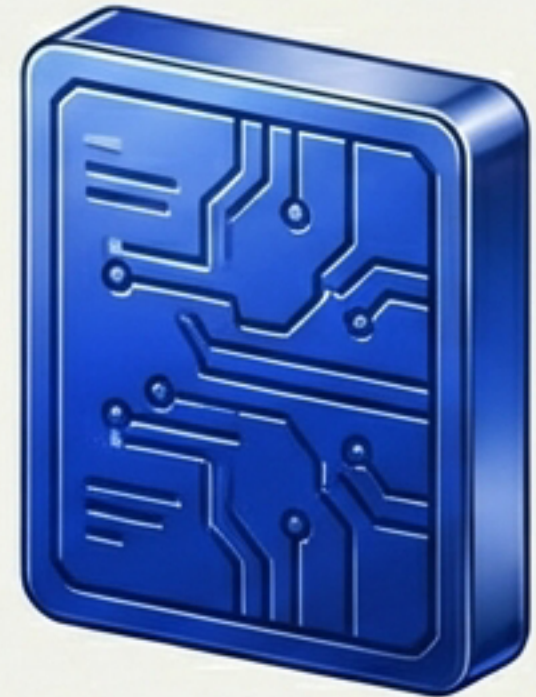
The Prompt-to-Code Pipeline



Prompt as Spec
Human Architect



LLM as Execution
Virtual Implementation



Code as Optimization
Hardened Implementation

We bridge the gap between intent and execution by treating the prompt as a compiling unit.

Key Decisions Log

ID	Decision	Rationale
LE1	LLM as execution engine	Enables immediate workflows before code exists.
LE2	Interface abstraction	Hides implementation details from caller.
LE3	Start with "Hello World"	Validate simple cases before scaling.
LE4	Role prompts as specs	Prompts define component behavior.
LE5	Gradual replacement	Swap to code as patterns stabilize.
LE6	Keep LLM for reasoning	Use the right tool for complex logic.
LE7	Prompts are testable	Validation via output comparison.